Built-in Functions

Introduction to Computer Programming

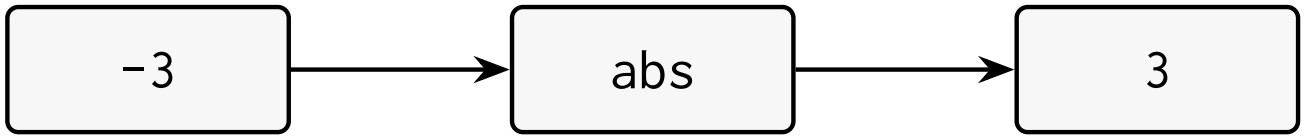
Chandra Gummaluru

chandra-gummaluru.github.io

A **function** is a named piece of code that does one job.

To use a function, we **call** its name and pass in any **arguments** (inputs).

```
function_name(argument, argument, ...)
```



`abs(-3)` evaluates to `3`; arguments are separated by commas

The function then **returns** its output.

A function call is an expression, just like any other expression.

Python comes with many **built-in functions** that are always available.

function	arguments	example	what it produces
`max`	2 or more	`max(3, 5, 4)`	the largest argument
`min`	2 or more	`min(3, 5, 4)`	the smallest argument
`abs`	exactly 1	`abs(-3)`	the absolute value
`pow`	2 or 3	`pow(3, 2)`	the first raised to the second
`round`	1 or 2	`round(4.6)`	the nearest number
`type`	exactly 1	`type(4.6)`	the argument's type

`dir(__builtins__)` lists them all; for now, look at the lowercase ones

builtins.py

Program

```
1
```

Shell

```
>>> max(3, 5, 4)
5
>>> min(12, 45, 76)
12
>>> abs(-56)
56
>>> pow(2, 5, 3)
2
>>> round(4.56789, 3)
4.568
>>> type(45)
<class 'int'>
```

`max` and `min` produce one of their **arguments**, so the type of the result is the type of the argument that was chosen.

```
>>> max(34.25, 564.34)
564.34
>>> max(34.25, 564)
564
>>> min(12, 45, 76)
12
>>> abs(-453.234)
453.234
```

`max(34.25, 564)` is the `int` `564`, not `564.0`

Without running the code: what value does `min(4, 6, 2.5)` produce, and what is its type?

- ✓ `2.5`, a `float`
- B `4`, an `int`
- C `2`, an `int`
- D `6`, an `int`

Without running the code: what value does `max(10.1, 13, 16)` produce, and what is its type?

- ✓ `16`, an `int`
- B `16.0`, a `float`
- C `10.1`, a `float`
- D `13`, an `int`

Without running the code: what value does `abs(-5.2)` produce, and what is its type?

A `-5.2`, a `float`

✓ `5.2`, a `float`

C `5`, an `int`

D `6`, an `int`

Without running the code: what value does `pow(2, 3)` produce, and what is its type?

A `6`, an `int`

B `8.0`, a `float`

✓ `8`, an `int`

D `9`, an `int`

help and optional arguments

The built-in function `help` describes how to call another function.

```
>>> help(pow)
Help on built-in function pow in module builtins:

pow(base, exp, mod=None)
    Equivalent to base**exp with 2 arguments or
    base**exp % mod with 3 arguments
```

An argument with = in the help is **optional**: if we leave it out, Python uses a default value.

Using the output of `help(round)` : what are the types function `round` can return?

```
>>> help(round)
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.  Otherwi
    the return value has the same type as the number.  ndigits may be nega
```

A only `int`

B only `float`

✓ `int` or `float`

D `str`

Using the output of `help(round)`: what is the *minimum* number of arguments `round` can take?

```
>>> help(round)
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.  Other
    the return value has the same type as the number.  ndigits may be ne
```

Answer: `1`

Using the output of `help(round)`: what is the *maximum* number of arguments `round` can take?

```
>>> help(round)
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.  Other
    the return value has the same type as the number.  ndigits may be ne
```

Answer: `2`

What value does this produce?

```
round(1.6)
```

Answer: `2`

What value does this produce?

```
1  round(3.14159, 2)
```

Answer: `3.14`

What value does this produce?

```
1  round(2.5)
```

- ✓ 2
- B 3
- C 2.5
- D 3.0

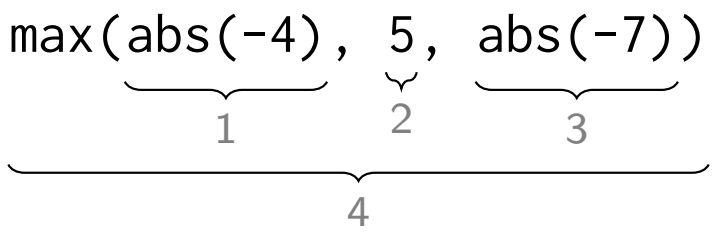
What value does this produce?

```
round(3.5)
```

Answer: `4`

nested function calls

An argument can be any expression, including another function call. To call a function, Python first evaluates its arguments, **left to right**.



(1) `abs(-4)` is evaluated, producing `4`

(2) `5` is evaluated, producing `5`

(3) `abs(-7)` is evaluated, producing `7`

(4) `max` is called with those values: `max(4, 5, 7)` produces `7`

Which function call is evaluated *first*?

```
1  pow(2, pow(pow(2, 1), 2))
```

A `pow(2, ...)`

B `pow(pow(2, 1), 2)`

✓ `pow(2, 1)`

D they all happen at once

What value does this expression produce?

```
pow(2, pow(pow(2, 1), 2))
```

Answer: `16`